

A Review of *Theory of
Computation via Inquiry-Based
Learning* by Iyad A. Ajwa

Nathaniel Miller

University of Northern Colorado

Abstract

In Fall 2017, I taught a senior capstone topics course out of the *JIBLM Notes Theory of Computation via Inquiry-Based Learning* by Iyad A. Ajwa. The notes were an excellent basis for an inquiry-based class in which some of the students were liberal arts math majors, while others were computer science and other applied emphasis math majors. While the course notes ask students for some proofs, there are also many problems that deal with more concrete constructions. This makes the notes suitable for a class of mixed proof-writing abilities. It also changes somewhat the way that a course taught from the notes needs to be run. The one main downside to the notes is that some of the definitions are not quite mathematically correct, making it hard for students to make sense out of them. This review discusses the strengths and weaknesses of the notes, and how I ended up using them in my class.

1 Introduction

In Fall 2017, I was scheduled to teach our upper-level topics course in the math department at the University of Northern Colorado. This course is a required capstone course for all of our liberal arts math majors and our applied math majors. The liberal arts math majors take a very traditional proof-oriented sequence of courses designed to prepare them for graduate studies in mathematics. The applied math majors take many fewer proof-oriented classes, and choose between concentrations in computer science, applied statistics, or general applied mathematics. Of these, the concentration in computer science is currently the most popular. I knew that I wanted to teach the class in an inquiry-based format, and that for this capstone course, I wanted to try using a fairly traditional Moore method style of class, in which students would solve problems on their own and present them at the board each day. (Almost all of my teaching is inquiry-based, but I have mostly used this particular traditional Moore method style of teaching in graduate courses.) I wanted to find a topic and a set of notes that would work on several different levels for the different types of students in the class, that would have some connections to computer science and programming for the computer science students in the class, and that would have a mixture of proofs and other kinds of problems. I settled on Iyad A. Ajwa's notes entitled *Theory of Computation via Inquiry-Based Learning* [1] as meeting all of these criteria and decided to use them for my class.

2 Teaching the class

My class consisted of twenty students. Of these, fourteen were applied math majors, four were liberal arts math majors, one was a secondary education math major, and one was a physics major. More than half of the applied math majors were concentrating in computer science. Fourteen of the students were seniors, five were juniors, and one was a sophomore. At least four of the graduating seniors who took the class have since gone on to graduate school.

The class was taught in a fairly typical modified Moore method style, as described, for example, in [2]. Students were expected to work through the problems and theorems in the text on their own. I would then call on students to present each problem in class, and we would discuss each solution as a class. Students always had the option of declining to present a problem when called on without losing any points. I kept a record of who presented each problem, and awarded presentation points for each presenta-

tion. In general, the first correct solution presented to a moderately difficult problem was worth 10 points, while other substantially different but also correct solutions that were presented were worth 5 points. 5 points were also sometimes awarded for the presentation of very easy problems, or for substantial attempts at harder problems that turned out to be not entirely correct. Students also received a participation grade that was determined by how actively and frequently they participated in class discussions.

Students were encouraged to work on problems together outside of class, as long as they were not just giving solutions to one another. They weren't allowed to use other outside references, however. On the first day of class, and at several other points in the semester when most people were stuck on a hard problem, we worked in groups in class to figure out the solutions. Students were expected to write up solutions to every problem in the text after they were presented in class (if not before). At the end of each section, I would collect all of the solutions and grade 2–3 of them at random. I also gave an in-class midterm and final that together accounted for about half of the students' grades. Many of the class policies from the syllabus can be found in Appendix A. The prerequisite for the course was one semester of discrete mathematics, which ensured that students had a least some experience with proof and with some similar topics.

I was generally happy with this choice of notes. The subject matter and the style of presentation worked well for this audience, particularly the liberal arts math majors and the computer science concentration math majors. It didn't work nearly as well for the applied statistics math majors, but this would have been an issue with almost any choice of topic, given the diverse backgrounds of the students in the class. The notes contain a lot of exercises that are not proofs, and many of the early proofs are routine enough that students who have seen a few should be able to do the others, even if their background in proof isn't particularly strong. For example, the first problems in the notes have to do with set identities. This was a great starting point, since there are several different but relatively routine ways to approach these, such as double set inclusion or using Venn Diagrams. We started out working in groups in class on the first day, and several students presented different ways to do the first couple of problems, which allowed almost everyone to figure out how to do the rest for the next class.

In general, the mixture of proof-oriented problems and other kinds of problems was just right for the class that I had. The hardest proofs were challenging even for the students going on to graduate school in mathematics, while at the same time, there were plenty of problems that asked the students to perform computations and to translate between different repre-

sentations using standard algorithms. I would have liked to have seen more problems of the form “create an automaton that accepts this given language,” which would allow students to use their programming skills in this environment. I added some problems like this to our class discussions and to the midterm and final. See Appendices B and C for the midterm and final that I gave.

3 Timing

My class started out going much slower than the author describes his class as going, but we sped up as the semester went on and were able to complete all but the last section of the notes (section 3.5). In the introduction for the instructor, Dr. Ajwa indicates that he is able to cover Chapter 1 of the notes in two weeks. However, at Ashland University, the prerequisite for this course appears to be two semesters of discrete math and an introduction to proofs course. At the University of Northern Colorado, we only offer one semester of discrete mathematics, which doubles as our intro to proofs class, and it doesn’t typically include any of the material from Section 1.5 on Alphabets and Languages. It therefore took us three weeks to get through the review material in sections 1.1–1.4, and then two more weeks to get through what was for us entirely new material in sections 1.5 and 2.1. Thus, our midterm only covered the material up to section 2.1. I do think that it is beneficial to start out slow in a course like this while the students are acclimating to the course expectations, so I didn’t mind spending this much time on the introductory material.

We then spent three weeks on the central material of sections 2.2 and 2.3; three weeks on sections 2.4 and 2.5; and the remaining four weeks of the course on sections 3.1–3.4. We didn’t cover section 3.5.

4 Specific details about using the notes

The biggest issue that I found when using these notes that required me to change my teaching a bit was the complexity of the definitions. Normally, when I teach using Moore method notes, I allow students to try to make sense out of the definitions on their own first, and then we discuss any questions that come up in class. However, the main definitions in Chapters 2 and 3 of these notes are too complicated for that. Instead, it became very important for me to discuss the definitions with the class and run through some examples before they tried to do any problems that relied on the defi-

nitions; otherwise, most students were not able to figure out the definitions well enough to make any progress at all.

As one example, the definition of a Nondeterministic Finite State Automaton (NFA) is given in Definition 17 of Chapter 2 in the notes, but really, to make sense of it, you need to understand everything up to Definition 27. So, when we were about to start section 2.2, we created an example of an NFA as a class, figuring out each piece of the quintuple that defines it, and then working our way through the subsequent definitions to see how they all work together. This worked well, and allowed the students to continue on in the notes.

Particular note should be taken of Construction 33 in Section 2.3 (NFA to DFA). Dr. Ajwa states in his introduction to the instructor that this construction “needs some attention. It is recommended that you as the instructor be involved in explaining the algorithm.” However, even this warning understates the amount of help that students will need with this construction. Part of the problem is that the description given in the notes leaves out so many crucial details that it isn’t really possible for students to understand on their own.

Step 1 of the construction makes sense, although it should mention that you have to repeat part (a) multiple times until Δ stops changing. Step 2, however, makes no sense as written. It refers to sets Q_i , which from context must be subsets of Q , but the notes never define what the Q_i are. Then, later in the construction, we are supposed to check if (Q_i, σ, Q_j) is in Δ , but elements of Δ are supposed to be in $Q \times (\Sigma \cup \{\lambda\}) \times Q$, not in $2^Q \times (\Sigma \cup \{\lambda\}) \times 2^Q$. Thus, there is virtually no way that a student (or an instructor, for that matter) could read this construction as written and successfully make sense out of it.

Theorem 35 gives an alternative construction of a DFA from a NFA in which the states of the DFA are elements of 2^Q . Also of note is the fact that Theorem 35 only works if the given NFA is λ -free (although this condition isn’t stated in the notes), while Construction 33 part 1 shows how to make an NFA λ -free. This gives a hint as to the intent of Construction 33 part 2: to produce the DFA of Theorem 35, but to construct it in stages in order to limit the states of the DFA to those that are actually reachable from the starting state.

One way to modify part 2 of Construction 33 to make it mathematically correct would be to rewrite it as follows:

2. Produce M' from M_1 by applying the following iterative Subset Construction Algorithm:

We define a sequence of subsets Q_i of Q , and for each put a corresponding state i into Q' .

Let $Q_0 = \{s\}$ be the zeroth subset of Q and let 0 be the corresponding state in Q' .

Set $i = 0$ and set $last = 0$.

While $i \leq last$ do:

For each $\sigma \in \Sigma$ do:

- Let $P_{i,\sigma} = \{p \in Q \mid \exists q \in Q_i \text{ such that } (q, \sigma, p) \in \Delta\}$.
- If there already exists some $k \leq last$ such that $P_{i,\sigma} = Q_k$ then set $j = k$; otherwise, set $last = last + 1$, set $j = last$, and set $Q_j = P_{i,\sigma}$.
- Set $\delta(i, \sigma) = j$.

Set $i = i + 1$.

Set $Q' = \{i \mid 0 \leq i \leq last\}$.

Set $F' = \{i \mid Q_i \cap F \neq \emptyset \text{ and } 0 \leq i \leq last\}$.

If I teach this course out of these notes again (as I hope to get to), I will give the students the corrected version of Construction 33, but I wouldn't expect most of them to be able to figure it out on their own until we had gone through several examples in class.

Theorem 35 and Exercises 38 and 39 are all slightly incorrect as stated in the notes. Theorem 35 is only true if the starting NFA is λ -free, while the statements being proven in Exercises 28 and 29 are only true if $Q_1 \cap Q_2 = \emptyset$. From my point of view this wasn't a problem, since I always tell my students to treat theorems stated in the notes as conjectures to be proven or disproven, so this was a nice opportunity for students to notice the issues that needed fixing. It is something that instructors will want to be aware of, though.

Remark 50 explains the next complicated algorithm introduced in the notes. Again, it is complicated enough that students will need help deciphering it. Step one also has a few mistakes as stated. It says "...if $\delta(q, s) \neq \emptyset$ for all q in Q , then ...," but δ is supposed to be a function from a state and a symbol to another state rather than from two states to a symbol, so this doesn't make sense. What the author intends is "...if there is a $q \in Q$ and a $\sigma \in \Sigma$ such that $\delta(q, \sigma) = s$, then ...". However, there is a larger issue here. In step 1, we start with a DFA and then add λ -transitions so that we have unique starting and finishing states. But according to the definitions adopted in the notes, λ -transitions are only available in NFAs, not in DFAs. So, really we need a step zero in which the given DFA is converted to an equivalent deterministic NFA, as described in Definition 30. Then λ -transitions can be added to this NFA as described in step one.

Step two of Remark 50 is the main part of the algorithm. It probably isn't intelligible to students without a good deal of additional explanation. (I had to look up the algorithm elsewhere in order to make sense of it, in order to then explain it to my students.) The idea behind the algorithm is that we are going to eliminate all of the states except for the starting and finishing states, one by one, preserving the information contained in all paths through the state when we remove it. One additional preparatory step is required: if there are multiple direct ways to get between two states, we have to combine them before doing step two. For example, if we have transitions (p, a, q) and (p, b, q) , we would combine them into a single transition $(p, (a + b), q)$. Next, we pick a state q to eliminate. We can pick any state other than the start or finish state. Before we can eliminate q , we must deal with all possible transitions to state q , a possible loop from q back to itself, and all possible transitions from q to another state. The diagrams in step two show how to deal with a transition from state p to state q , followed by a loop from state q back to itself (which can be followed any number of times), followed by a transition from q to another state r . What isn't explicitly stated is that we must do this for all possible combinations of transitions to and from q . After we've done this for all of these combinations, state q is no longer needed, and we can remove it along with all transitions to and from it. We then repeat this process with all of the other nodes, one at a time, until we are left with a single transition from the start to the finish, labeled by a single regular expression that describes the language of the original DFA. Note that according to the notation introduced in Definition 46, the red label on the transition from p to r should really be $E_0 + (E_1 E_2^* E_3)$.

This all sounds very complicated, but it isn't very hard to show students how it works, which is why it makes sense for the instructor to go through the algorithm with the students before asking them to do it on their own. I also realized after collecting Exercise 51 that the regular expression you get depends very much on the order you choose to eliminate the states, making it tedious to check if a student's answer is correct. I would therefore highly recommend specifying the order in which states should be eliminated on any problems that are going to be collected in order to make the grading easier.

Lemma 53, the Regular Languages Pumping Lemma, was really good for my students to look at. They needed a lot of help with it, so we spent a whole class period working in groups to

1. make sense out of what the pumping lemma means;
2. figure out how to prove it; and

3. figure out how to use it for the proofs in Exercise 54.

Students were eventually able to understand the proof of the pumping lemma, which really drove home the usefulness of the connection between DFAs and regular languages. Furthermore, once they had seen one proof using the pumping lemma, the other proofs were similar enough that almost everyone was able to figure out how to get started on the other proofs, while at the same time, they were different enough not to be completely rote. Thus, they seemed to be at exactly the right level for most of my students to have productive struggles working on them.

Moving on to Chapter 3, the first thing to be aware of is that the remarks about Chapter 3 in the Introduction To the Instructor appear to have been written with a previous iteration of the notes in mind, and the numbering in the introduction doesn't match the numbering in the chapter. The first comments are about "Exercise 24"; as far as I can make out, they refer to an exercise that was removed in this version of the notes. They don't appear to make sense for any exercise currently in Chapter 3. The next comment is that "The proof to Theorem 20 uses the construction proofs in Exercises 17, 18, and 19." This should really read, "The proof to Theorem 21 uses the construction proofs in Exercises 18, 19, and 20." Next, Dr. Ajwa states that "[t]he proof to Theorem 21 is beyond the scope of this course." He really means Theorem 22, not Theorem 21. This is crucial to be aware of, because in the notes in Chapter 3, there is no indication that Theorem 22 is to be treated any differently than any other theorem. Therefore, you will need to warn your students well in advance of the time they get to it that Theorem 22 is there to be used, but that they shouldn't try to prove it themselves.

Two sentences later, the introduction refers to "Exercise 22" and "Theorem 21"; these should really be "Exercise 23" and "Theorem 22." The rest of the paragraph is numbered correctly, except that every time "Theorem 21" is mentioned, it should be "Theorem 22" instead.

When studying section 3.2 on parsing, my class spent some time looking at grammars found online for several common programming languages that the students were familiar with (C, python, java, and scheme). I found that a formal grammar for almost any programming language can be found by googling the name of the language followed by the word "grammar." The students really liked looking at these, and I think it made this material seem more relevant to them.

As previously noted, Theorem 22 isn't intended to be proven by the students. However, either here or after the following section on pushdown automata, it would be good to talk to them about the connection between context free grammars and pushdown automata (they define the same lan-

guages), and about how we can use this connection to prove Theorem 22. This connection will remain unproven, but it is very natural and easy for the students to believe it given the similar connection between DFAs and regular languages proven in the previous chapter. This also makes it clear why Chapter 3 is about these two otherwise seemingly unrelated topics.

Finally, we note that for the Push operation on pushdown automata to work as described in Definition 27, we must change Definition 25 slightly to allow reading λ from the stack, so that we should have

$$\Delta \subseteq (Q \times (\Sigma \cup \{\lambda\}) \times (\Gamma \cup \{\lambda\})) \times (Q \times \Gamma^*).$$

5 Conclusion

All in all, I enjoyed using these notes. Automata Theory is a great topic for an IBL course, and it is one that may appeal to a different set of students than many other IBL notes. These particular notes have a nice mixture of proofs and more concrete constructions and exercises that make them well suited for use with a class with a wide variety of backgrounds. My only reservation about these notes is that there are a number of places where they aren't mathematically precise, which may have the unfortunate side effect of reinforcing the feeling that some students have that they can't make sense of mathematical definitions on their own. Several of the students specifically commented on their anonymous course evaluation forms at the end of the semester that they found the definitions in the notes to be very confusing and hard to make sense of. However, I think this issue can be somewhat ameliorated by careful attention from the instructor. I thought that the course I taught from these notes went well, and I would encourage others to try using them.

References

- [1] Ajwa, I. A. (2012). *Theory of Computation via Inquiry-Based Learning. Journal of Inquiry-Based Learning in Mathematics*, Course Notes No. 25.
- [2] Coppin, C. A., Mahavier, W. T., May, E. L., & Parker, G. E. (2009). *The Moore method: A pathway to student-centered instruction*. Washington, DC: Mathematical Association of America.

- [3] Hopcroft, J.E. and Ullman, J.D. (1979). *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley Series in Computer Science and Information Processing.
- [4] Rodger, S.H. and Finley, T. W. (2006). *JFLAP: An Interactive Formal Languages and Automata Package*. Sudbury, MA: Jones & Bartlett Publishers.

A Sample Syllabus Policies

Course Description: An introduction to the logical and mathematical foundations of computation. Topics discussed will include finite-state automata and context-free languages. The course material will be presented in an inquiry-based context in which students will solve problems and prove theorems and present their solutions to the class.

In this course, we will be following a method of learning mathematics pioneered by the mathematician R. L. Moore at the University of Texas at Austin, and sometimes referred to as “The Modified Moore Method.” The hallmark of this method is that the students (you!) are responsible for creating all of the mathematics we will be discussing. This will allow you to develop your own deep conceptual understanding of the course material.

The core of this strand is the sequence of definitions, problems to be solved, and theorems to be proven, all of which can be found in the course notes. You will be solving the problems and proving the theorems, and then presenting your solutions to the class. Here is how this part of the class will work:

1. Presentation points will be awarded for solving Problems and proving Theorems and then presenting the solutions to the class.
2. Problems and Theorems should be viewed as conjectures—that is, they may be either true or false. They can be solved by showing which category they fall in. In other words, you solve a problem or theorem by giving a convincing argument that it is true (or false). If the Problem or Theorem is not correct as stated, you should try to see if there is a way to fix it.
3. You will earn presentation points by presenting a correct solution to an unsolved problem to the class. A solution is correct if it is unanimously judged by the class (including me) to be completely convincing. A correct solution presented to the class normally earns 10 points, although solutions may be worth fewer or more points for problems that are easier or harder than average.
4. Presentation points may be awarded for other reasons, such as for contributing a significantly different alternative proof, or proposing a conjecture that gets made into one of the course problems.
5. The intent of this course is for you to solve the problems yourself. You may discuss the problems you are working on with other members of the class as long as neither of you has already solved the problem. If

you have solved a problem you can try to help another student arrive at the solution themselves, but you MAY NOT just give them your solution.

6. We will start presentations each day from where we left off the day before. It is up to you to judge how far we are likely to get each day and to keep far enough ahead in the problem sequence that you will be prepared for any problems or theorems you are called on to present.
7. You will also need to keep a solutions journal in which you write up your solutions to every problem from the textbook. I will periodically collect your journals and randomly pick a few problems to grade.
8. I encourage you to type up all of your solutions in Microsoft Word or in the mathematical typesetting system LaTeX, though scans of handwritten solutions will also be accepted. However, typing your solutions will make it much easier to revise them after they have been presented in class. I will be collecting your solutions journals periodically and awarding homework points for them. Some problems may not be presented in class, and only turned in through your journals. However, any problem that any of us would like to see discussed in class will be discussed in class.
9. Each time I collect your solutions journal, I will pick approximately two problems at random to grade. Thus, you need to make sure that all of your solutions are written up to the quality of something you are turning in to be graded. Since you will be turning in problems after they have been discussed in class, you should be able to do an excellent job of writing them up. Problems that are completely clear and correct will be awarded full credit, and problems that are not completely clear and correct will have points deducted as appropriate. For example, if a problem is worth 10 points, one point might be deducted for a solution with minor typos, misstatements, or missing explanation; two to three points might be deducted for a minor conceptual flaw in the reasoning or significant missing explanation; five to eight points might be deducted for major conceptual flaws in the reasoning and explanation; and all ten points will be deducted for problems that are completely missing or contain no significant correct work.s
10. The only texts that you may consult in solving these problems are the official course materials and your own notes. You should not consult any outside textbooks or other materials unless I have specifically given you permission. Solutions to some of the problems that we will be covering can be found in other textbooks, online, *etc.*, but if you

consult these books, you will be depriving yourself of the understanding that will come from solving the problems yourself.

11. Class presentation grades will be determined by how many presentation points you have at the end of the course. I can't tell you in advance how many points will correspond to a certain grade, but if I feel that anyone with a certain number of points has been doing A level work in their presentations, for example, then everyone with at least that many points will get an A for their presentation grade.
12. In addition to class presentations, we will have midterm and final exams.

Grades: Grades will be determined as follows:

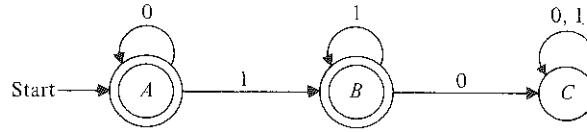
1. 35% in-class participation and presentations
2. 20% written homework
3. 20% midterm exam
4. 25% final exam

B Sample Midterm

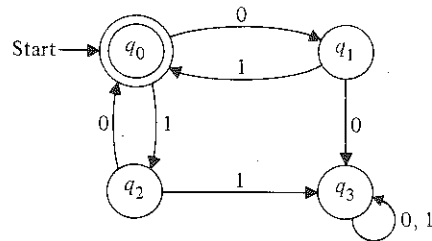
Some of the problems on this sample midterm are taken from or modified from exercises in Hopcroft and Ullman [3].

1. For each of the following set equations, either show that it is always true, or else give a counterexample showing that it is not.
 - (a) $A - (B \cap C) = (A - B) \cup (A - C)$
 - (b) $(B - A) \cup (B - C) = B - (A \cup C)$
2. Let $\Sigma = \{0, 1\}$. For each given language over Σ , draw the state diagram of a DFA that accepts that language.
 - (a) $L_1 = \{\lambda, 0, 00, 001\}$
 - (b) $L_5 = \{w \in \Sigma^* \mid |w| \geq 2 \text{ and } w \text{ begins and ends with } 1\}$
 - (c) $L_{15} = \{w \in \Sigma^* \mid |w|_0 \text{ is even}\}$
 - (d) $L_{15} = \{w \in \Sigma^* \mid |w|_0 \text{ is even and } |w|_1 \text{ is also even}\}$
3. For each of the given DFAs, describe in words or symbols the language that it accepts.

(a)



(b)



4. Show that λ is a substring, a prefix, and a suffix of any string w .
5. Prove that for any two strings w and x , $(wx)^R = x^R w^R$.
6. (extra credit) A farmer wants to cross a river and take with her a wolf, a goat, and a cabbage. She has a boat that can fit herself plus either the wolf, the goat, or the cabbage. If the wolf and the goat are alone on one shore, the wolf will eat the goat. If the goat and the cabbage are alone on the shore, the goat will eat the cabbage.

Let $\Sigma = \{A, W, G, C\}$. Design a finite automaton that accepts a string from Σ^* if and only if it represents a solution to this problem, where A represents the farmer crossing alone, W represents a crossing with the wolf, G a crossing with the goat, and C a crossing with the cabbage. Use your automaton to find two solutions to the problem that are substantially different from one another.

C Sample Final

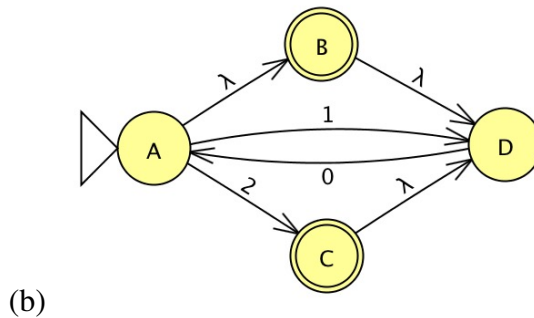
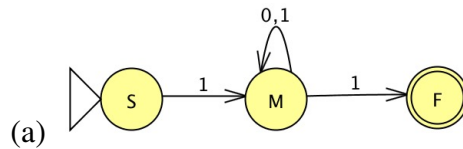
All of the diagrams in this final were created using the Java Formal Language and Automata Package (JFLAP) [4], which is freely available online at <http://www.jflap.org>, and which I found very useful in creating diagrams for this class.

1. For each of the following set equations, either show that it is always true, or else give a counterexample showing that it is not.

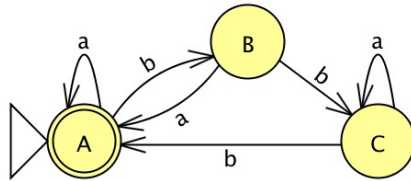
(a) $(A \cup B) \cap C = (A \cap C) \cup (A \cap B)$

(b) $(A \cap B) \cup C = (A \cup C) \cap (B \cup C)$

2. For each NFA, use the algorithm from the notes to find an equivalent DFA.



3. Use the algorithm discussed in the class notes to find a regular expression that describes the language of this DFA. Show your work.



4. State the pumping lemma and explain why it is true.
5. Let $\Sigma = \{0, 1\}$. For each given language over Σ , either show that it is regular, or else show that it is not regular.

(a) $L_1 = \{0^{2m} \mid m \geq 0\}$

(b) $L_3 = \{0^m 1^k \mid 0 \leq m \leq 3, k \geq 0\}$

$$(c) L_4 = \{0^{2m}1^m \mid m \geq 0\}.$$

6. Let $G = (V, \Sigma, R, S)$ be the CFG with $V = \{S, A, B, a, +, *, (,)\}$, with $\Sigma = \{a, +, *, (,)\}$, and where R contains the following rules:

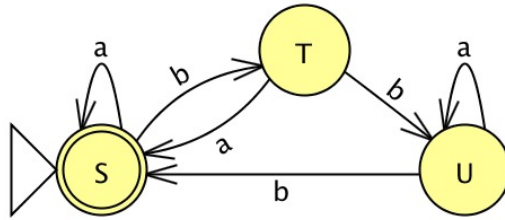
$$S \rightarrow S + A \mid A$$

$$A \rightarrow A * B \mid B$$

$$B \rightarrow a \mid (S)$$

Show the parse tree in G of the string $(a + a) * (a + a * a)$.

7. Construct a CFG that accepts the language $L_4 = \{0^{2m}1^m \mid m \geq 0\}$.
8. Construct a CFG that generates the language accepted by the following DFA:



9. Let $M_1 = (Q_1, \Sigma, \Delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \Delta_2, s_2, F_2)$ be NFAs, with $Q_1 \cap Q_2 = \emptyset$. Explain how to create an NFA M such that $L(M) = L(M_1) \cup L(M_2)$.
10. Let $M = (Q, \Sigma, \Gamma, \Delta, q_0, F)$ be the pushdown automaton where $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a\}$, $F = \{q_0, q_2\}$, and Δ consists of the following transitions:

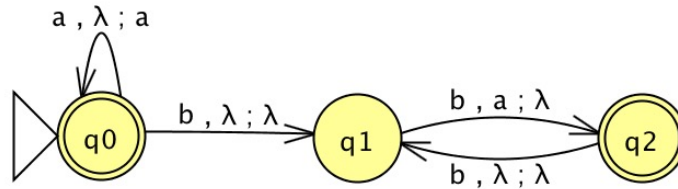
$$((q_0, a, \lambda), (q_0, a))$$

$$((q_0, b, \lambda), (q_1, \lambda))$$

$$((q_1, b, a), (q_2, \lambda))$$

$$((q_2, b, \lambda), (q_1, \lambda))$$

M can be illustrated as follows:



Explain what language is accepted by M , and why.

11. (extra credit) Construct a CFG that generates the language that consists of all regular expressions over the alphabet $\{0, 1\}$. For example, its language would include $1(11^*0 + 0^*1 + \Lambda)$.